Curtin College IT

Adapted from Curtin University — Discipline of Computing

Unix & C Programming (UCP1000)

Trimester 1, 2024

Assignment One

Weight: 15% of the unit

1 Introduction

Your task for this assignment is to design, code and test a game inspired from a classical puzzle game "Laser Tank" using the C programming language (in the C89 Standard). In summary, your program will:

- Read the parameters from command line and utilize them for the game configuration.
- Create dynamically-allocated 2D char array to make a simple ASCII-based game.
- Receive user input to control the flow of the game.
- Write a suitable makefile. Without the makefile, we will assume it cannot be compiled.

2 Code Design

You must thoroughly document your code using C comments (/* ... */). For each function you define and each datatype you declare (e.g. using struct or typedef), place a comment immediately above it explaining its purpose, how it works, and how it relates to other functions and types. Collectively, these comments should explain your design.

Your code should be structured in a way that each file has a clear scope and goal. For example, "main.c" should only contain a main function, "game.c" should contain functions that handle and control the game features, and so on. Functions should be located in reasonably appropriate files and you will link them using the makefile. **DO NOT** put everything together into one single source code file. Make sure you use the header files and header guards correctly. Never include .c files directly.

Make sure you free all the memory allocated by the malloc() function. Use valgrind to detect any memory leak and fix them. Memory leaks might not break your program, but penalty will still be applied if there is any. If you do not use malloc, you will lose marks.

Keep in mind that it is possible to lose significant marks with a fully working program that is written messily, has poor structure, contains memory leaks, or violates many of the coding standards.

3 Academic Integrity

This is an assessable task, and as such there are strict rules. You must not ask for or accept help from anyone else on completing the tasks. You must not show your work at any time to another student enrolled in this unit who might gain unfair advantage from it. These things are considered plagiarism or collusion. To be on the safe side, never ever release your code to the public.

Do NOT copy C source code from internet resources or artificial intelligence code generators. Copying code from the internet without referencing will be considered plagiarism. If you include code from other sources with a valid reference, then that part of the code will not be marked since it is not your work.

Staff can help in understanding the unit material in general, but nobody is allowed to help you solve the specific problems posed in this specification. The purpose of the assignment is for you to solve them on your own, so that you learn from it. Please see Curtin College's Academic Integrity policy for information on academic misconduct (which includes plagiarism and collusion).

You will be required to attend quick interview to answer questions about your program. In order for your work to be graded you will be required to explain how it works. Any code that you cannot adequately explain to your tutor will not receive marks and may be referred to as evidence in an Academic Misconduct inquiry.

4 Task Details

4.1 Quick Preview

You will implement a simple game inspired from a classical puzzle game "Laser Tank". Further details on the specification will be explained on the subsequent sections.

4.2 Command Line Arguments

Please ensure that the executable is called "laserTank". Your executable should accept eight (8) command-line parameters/arguments:

```
laserTank <row_size> <col_size> <player_row> <player_col>
<player_direction> <enemy_row> <enemy_col> <enemy_direction>
```

- <row_size> and <col_size> determines the size of the playable map. The minimum and maximum value for <row_size> and <col_size> are 5 and 25 respectively. You need these numbers for dynamic memory allocations of 2D char array. (use malloc() function)
- <player_row> and <player_col> are the coordinates of the player tank. You can use these numbers as the array index of the map. Coordinate <0,0> is located at top left of the map.
- <player_direction> is the direction the player is facing upon starting the game. There are 4 choices: n,s,e,w (lowercase letter) which is derived from the words north, south, east and west.
- <enemy_row>, <enemy_col>, and <enemy_direction> have the same meaning as the previous

explanations. However, these arguments configure the enemy tank.

This is one example: `./laserTank 10 20 1 1 w 5 10 n`

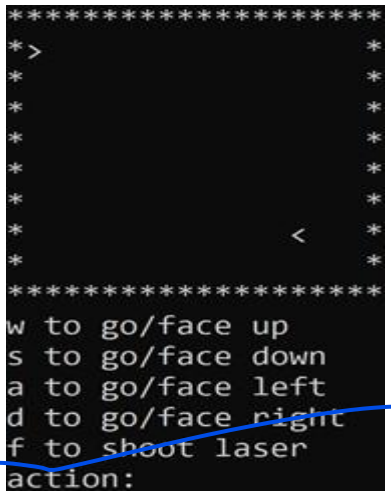
Note: Remember, the name of the executable is stored in variable argv[0].

2.1 Map

The map can be derived from a 2D char array, which has been dynamically allocated based on the command line arguments.

2.2 Main Interface

Once you run the program, it should clear the terminal screen and print the map along with the instructions:



This is the interface where the user plays the game. The user can type the command after the sentence "action:". Every time the user inputs the command, the program should update the map accordingly and refresh the screen. (clear the screen and reprint everything)

2.3 User Input

The user only need to type 1 character for each command (lower case). Here is the list of the possible commands:

- **'w'** moves the player one block above. If the player tank is not facing upwards, this command will only cause the tank to face upwards.**'s'** moves the player one block below. If the player tank is not facing downwards, this command will only cause the tank to face downwards.**'a'** moves the player one block to the left. If the player tank is not facing leftwards, this command will only cause the tank to face leftwards.
- **'d'** moves the player one block to the right. If the player tank is not facing rightwards, this command will only cause the tank to face rightwards.
- **'f'** causes the player tank to shoot the laser on the direction it is facing. Please refer to the next section on "Laser Animation" for more details. Additionally, please watch the supplementary video on "Assignment Demonstration".
- Any other character will be ignored. The program still need to refresh the interface to

enter new command. You can add warning message if you want.

> **Note:** If the player attempts to move to the same location as the map border OR the enemy tank, it should not do anything. (player location stays the same)

2.4 Laser Animation

When the user provides the command 'f', there will be an additional character representing the laser on the map. You can use the character '-' if the tank is facing horizontal direction, or the character '|' if the tank is facing the vertical direction. When the laser is being shot, the program should trigger a simple animation of the laser moving from the front of the player tank. The laser will keep moving forward until it hits the border OR the enemy tank. When the laser hits the enemy tank, its character should be replaced with 'X'.

In order to make a simple animation, you can use the UCPSleep function provided in Moodle. The idea is to move the position of the laser one block at a time with a very brief sleeping period in between. We recommend sleeping time less than 0.5 seconds for reasonable speed. When the animation is still running, the program should NOT show the game instructions and NOT prompt the user for any command.

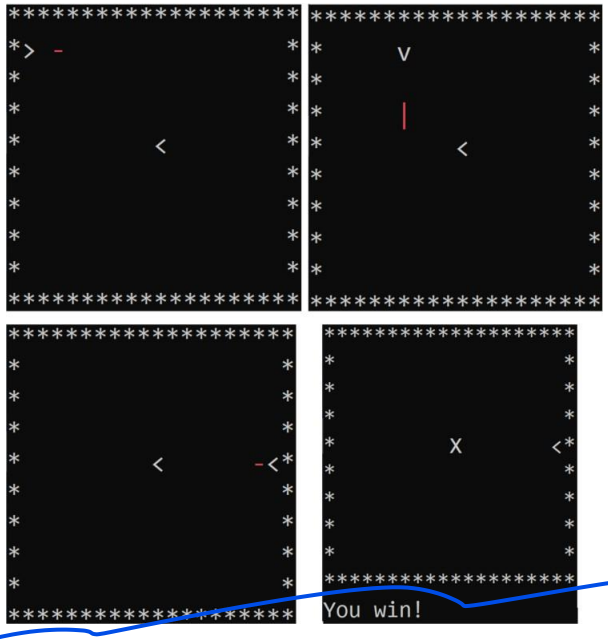
The laser should be printed on the map using a font color other than the default color.

> **Note:** If the enemy tank is exactly in front of the player (no distance), then there is no need to show the laser, and the enemy tank should get destroyed immediately and the game should conclude.

2.5 Winning/Losing Conditions

The game will continue until the player either win or lose. This is the condition for each result:

- **WIN**: The player tank manage to hit the enemy tank with the laser. This can only happen when the player tank shoots the laser while facing the enemy tank, regardless of the distance between them. The enemy tank's character will change into a character 'X' (uppercase X).
- **LOSE**: The player moves in front of the enemy tank (facing the player). The enemy tank will immediately shoots the laser to the player (with the laser animation). The player cannot move when the animation is happening. The player's character will change into a character 'X'.



When this occurs, please do not forget to **free** all the memories allocated via malloc() function before the program ends.

2.6 Makefile

You should write the makefile according to the format explained in the lecture and practical. It should include the Make variables, appropriate flags, appropriate targets, and clean rule. We will compile your program through the makefile. If the makefile is missing, we will assume the program cannot be compiled, and you will lose marks.

2.7 Assumptions

For simplification purpose, these are assumptions you can use for this assignment:

- The coordinates of the player tank and enemy tank will not overlap from the provided command line arguments. But, you still need to check whether the coordinates are inside the map boundary.
- The coordinate of the tanks will not overlap with the border of the map as well.
- The player tank will NOT be in front of the enemy tank immediately upon starting the game. (NO instant lose)
- Any command or argument is always on correct datatype and in lower case (for letters).

3 Marking Criteria

This is the marking distribution for your guidance:

Code Structure:

- Contains any NON-C89 syntax. Compile with -ansi and -pedantic flags to prevent losing marks on this. **(5 marks)**
- Properly structured makefile **(10 marks)**
- Program can be compiled with the makefile and executed successfully without immediate crashing **(5 marks)**

- Usage of sufficient in-code commenting **(5 marks)**
- The whole program is readable and has reasonable structure, Multiple files are utilized **(5 marks).**
- Avoiding bad coding standard. **(10 marks)**

  Some of the most common mistakes are:

  - Using global variables
  - Calling exit() function to end the program abruptly
  - Using "break" not on the switch case statement
  - Using "continue"
- No memory leaks **(10 marks)**
- Please use valgrind to check for any leaks. If your program is very sparse OR does not use any malloc(), you will get zero mark on this category.

Functionality:

- Able to read and interpret command line arguments correctly. This includes clarifying the argument amount and values. If there is any issue, then the error message is printed on terminal and end the program. Otherwise, the map array should be allocated accordingly. **(10 marks)**
- Correct characters usage for the map and its components. **(5 marks)**
- Player tank moves and changes direction correctly based on player command. **(10 marks)**
- The laser is animated correctly on the map with the UCPSleep function. **(5 marks)**
- (BONUS mark) The laser is changing color every time it moves by one block. (hint: use static variable to keep track.) **(5 marks)** (total mark is still capped at 100 marks.)
- Able to refresh the screen with the updated map/instructions visualization every time the user enters the command. **(5 marks)**
- Winning/Losing condition is implemented correctly. (make sure to free the memory before wrapping up the program) **(10 marks)**

4 Short Report

If your program is incomplete/imperfect, please write a brief report explaining what you have done on the assignment. This report is not marked, but it will help us a lot to mark your assignment. Please ensure your report reflects your work correctly (no exaggeration). Dishonest report will lead to academic misconduct.

5 Final Check and Submission

After you complete your assignment, please make sure it can be compiled and run on a Linux-based operating system. If you do your assignment on other environments (e.g on Windows operating system), then it is your responsibility to ensure that it works in a linux environment. Ubuntu will be used for testing. In the case of incompatibility, it will be considered "not working" and some penalties will apply. You have to submit a tarball containing:

- **Your essential assignment files** – Submit all the .c & .h files and your makefile. Please

do not submit the executable and object (.o) files, as we will re-compile them anyway.

- **Brief Report** (if applicable) - If you want to write the brief report about what you have done on the assignment, please save it as a PDF or TXT file.

The name of the tarball should be in the format of:

<student-ID>_<full-name>_Assignment1.tar.gz
Example: 12345678_Antoni-Liang_Assignment1.tar.gz

Please make sure your submission is complete and not corrupted. You can re-download the submission and check if you can compile and run the program again. Corrupted submission will receive instant zero. You can submit it multiple times, but only your latest submission will be marked.

End of Assignment